bashagent: klartext till bash, utan att köra kommandot

Rebecka Vannerberg · bash/Linux · 19 oktober 2026

Verktyget tar en beskrivning på svenska och ger tillbaka tre saker: kommandot, en förklaring av vad delarna gör, och en riskmärkning. Kommandot skrivs ut och kan läggas i urklipp. Det körs inte.

Syfte

Att översätta en avsikt, till exempel ”hitta alla pdf:er större än 10 MB”, till rätt `find`-anrop kräver flaggor man sällan minns. En språkmodell klarar ofta den översättningen. Den är sämre på att veta när den har fel, och ett felaktigt `find … -delete` är dataförlust, inte ett syntaxfel.

`bashagent` lämnar därför kommandot till användaren. Två skäl bär den designen. Det första är att människan läser kommandot och förklaringen innan något händer: det finns ingen bekräftelsedialog att klicka bort, eftersom det enda sättet att köra är att själv klistra in och trycka Enter. Det andra är skaltillståndet. Ett program som kör `cd`, `export` eller `source` ändrar tillståndet i en subprocess som sedan dör. Användarens prompt står kvar. Genom att lämna över strängen till användarens eget skal fungerar de kommandona på samma sätt som `ls`.

Frågan mätningen ställer är om den lilla modellen räcker jämfört med den stora, på ett fast testset, när skillnaden bedöms mot en gräns som skrevs ner innan körningen. Urklipp är inte poängen med projektet. Den nischen är redan upptagen, vilket konkurrentanalysen tar upp sist.

Upplägg

Mätningen gjordes 19 oktober 2026. Fyrtio uppgifter, plattform GNU, temperatur 0. Modellerna är `gpt-4.1-mini` och `gpt-4.1`, samma leverantör, samma prompt och samma JSON-schema, så det som skiljer dem åt är storlek och pris. Hypoteserna skrevs samma dag, innan något mätanrop, och ligger i `eval/hypotheses.md`.

Testsetet är sammansatt så här:

Kategori	Antal	Jämförelse
Filsökning	8	Beteende, utdata
Textbearbetning och filtrering	7	Beteende, utdata
Katalog- och diskanalys	5	Beteende, utdata
Behörigheter och ägarskap	4	Beteende, filsystemstillstånd
Arkiv och komprimering	3	Beteende, filsystemstillstånd
Processer och system	4	Text mot facit
Git	3	Text mot facit
Nätverk	2	Text mot facit
Destruktiva kommandon	4	Text mot facit, körs aldrig

Tjugosju av fyrtio uppgifter är tänkta att jämföras på beteende, tretton mot en lista av accepterade varianter. Läsande uppgifter kördes i varsin Debian-container och utdata jämfördes. Destruktiva kommandon kördes inte, inte ens i containern. Övriga jämfördes som text mot facit. Ett likvärdigt kommando med annan stavning räknas därför som avvikelse i totalsiffran.

Containern utgår från `debian:bookworm-slim` med coreutils, findutils, grep, gawk, sed, tar och gzip. Alpine valdes bort eftersom BusyBox har en tredje syntax. En container i taget, med `--memory=512m --pids-limit=256 --cpus=1 --network=none`. Fixturen ligger på containerns skrivbara lager, inte på tmpfs. Gränsen kommer av maskinen: 8 GB totalt, och Docker har ungefär 3 GB att röra sig med.

Riskmärkningen har tre nivåer. `safe` läser eller listar. `caution` skriver eller ändrar något som är lokalt och i huvudsak reversibelt. `destructive` är irreversibel förlust, systempåverkan eller körning av ohörd kod. Modellen sätter en nivå. Regelverket i `risk.py` kan höja den, aldrig sänka den. Ett falskt larm kostar en läsning. Ett missat `rm -rf` kostar data.

Facit skrevs innan modellerna kördes. Utkasten i repot är handskrivna. Fältet `authoring_model` är `none`, så de kommer inte från någon av de två modeller som mättes. Planen ville att utkasten skulle komma från en tredje modell. Det är inte vad som står i testsetet. Ett beteendefacit räknas som riktigt när det ger det avsedda utfallet i fixturen. Destruktiva uppgifter och variantuppgifter kördes inte, så de faciten vilar på att flaggorna är lästa, inte på en körning. Huvudsiffran gäller GNU/Linux. Ett BSD-delprov ingick i planen och kördes inte i den här mätningen.

Vid n = 40 är ett 95-procentigt Wilson-intervall kring 75 % ungefär ±13 procentenheter. En skillnad under ungefär 15 procentenheter är inte statistiskt säkerställd. Den gränsen skrevs in i planen innan resultatet fanns. Inget konkurrerande verktyg publicerar en utvärdering, så det finns ingen extern siffra att ställa resultatet mot. NL2Bash (omkring 9 000 par, tre granskare) och InterCode-Bash (200 uppgifter i fyra filsystem) är andra uppgiftsmängder och andra mätetal. Beteendejämförelsen, utdata plus filsystemsavtryck, är lånad från InterCode-Bash. Den sker i testriggen, aldrig i produkten.

Resultat

	gpt-4.1-mini	gpt-4.1
Höll på det som jämfördes	35/40	33/40
Nya anrop	35	35
Från cache	5	5
Betalt den här körningen	0,012 USD	0,062 USD

Tillsammans med rökkörningen ligger hela API-kostnaden runt 0,09 USD. Den lilla modellen kostade ungefär en femtedel av den stora.

Skillnaden är fem procentenheter, och den billiga modellen ligger före. Fem procentenheter ligger under gränsen på ungefär 15. Skillnaden mellan modellerna är inte statistiskt säkerställd. Det är ett giltigt resultat: på det här testsetet räcker den billiga modellen, och det högre priset gav inte en högre totalsiffra.

Siffran är andelen uppgifter som höll på det som jämfördes. För de läsande uppgifterna är det utdata i containern. För övriga är det text mot facit, inklusive annan stavning. Precision, recall och latens finns definierade i planen och redovisas inte här, eftersom mätningen inte har de siffrorna. Ingen uppdelning per kategori finns i resultatnoten, så en sådan tabell redovisas inte.

Felanalys

Totalsiffrorna 35/40 och 33/40 räknar varje textavvikelse. Två avvikelser ändrar vad kommandot gör eller hur det bedömdes. Fyra andra är samma kommando med annan stavning.

Avvikelser som betyder något

`gpt-4.1` fick uppgiften ”lista katalogerna direkt under aktuell katalog” och skrev `find . -maxdepth 1 -type d`. Containern räknade det som fel, eftersom `.` själv kommer med. Facit har `-mindepth 1`.

`gpt-4.1-mini` fick uppgiften ”lista vanliga filer som ligger direkt i docs” och skrev `ls -p docs | grep -v /`. Risknätet såg snedstrecket och höjde märkningen till `destructive`, så kommandot kördes inte. Det är ett falskt larm i nätet, inte ett farligt kommando. Regelverket får bara höja märkningen, och det är därför ett läsande kommando stoppades: mönstret såg ut som en sökväg.

Samma kommando, annan stavning

De här texavvikelserna gör samma sak. De ska inte redovisas som att modellen hade fel.

- `ps -u $(whoami)` mot `ps -u "$USER"`
- `cmd=` mot `args=`
- `wget` mot `curl -O`
- `> notes.txt` mot `truncate -s 0`

Strängjämförelsen har redan räknat in dem i 35/40 och 33/40. Resultatnoten säger inte hur många av avvikelserna som är av det här slaget, så ingen justerad träffsäkerhet räknas fram här. Paret med `> notes.txt` och `truncate -s 0` är dessutom den stora modellens svar på truncate-uppgiften. Effekten är densamma, och båda märktes som `destructive`.

Den lilla modellen flaggade `truncate -s 0`, `find -delete`, `git clean` och `rm`. Det är kommandon som går runt en blocklista som bara letar efter `rm` och `dd`. Utfallet säger något om blocklistan, och det tas upp i konkurrentanalysen.

Jämförelse med hypoteserna

Hypoteserna skrevs innan något mätanrop. De som inte höll står kvar.

1. På enkla kommandon, ett verktyg och få villkor, är den lilla och den stora modellen ungefär lika träffsäkra. Det stämmer. Båda klarade i stort sett alla `find`-uppgifter.
2. På sammansatta kommandon, flera steg eller ett git-kommando där flaggan avgör om arbete förstörs, tappar den lilla och den stora klarar fler. Det stämmer inte på totalsiffran. Den lilla höll 35, den stora 33. En uppdelning på sammansatta uppgifter finns inte i resultatnoten, så hypotesen bedöms på den siffra som finns.
3. Totalskillnaden blir liten, under ungefär 15 procentenheter, vilket 40 uppgifter inte kan skilja från brus. Slutsatsen blir att den billiga modellen räcker. Det stämmer. Fem procentenheter, och den billiga var före.
4. Den lilla modellen missar farliga kommandon som inte ser farliga ut, som `find -delete` och `truncate`. Den stora flaggar fler. Det stämmer inte här. Den lilla flaggade `truncate -s 0`, `find -delete`, `git clean` och `rm`. Den stora skrev `> notes.txt` i stället för `truncate`, samma effekt, och det flaggades också som `destructive`.
5. Båda modellerna föreslår ibland GNU-flaggor som inte finns på macOS. Den lilla gör det oftare. Det är inte mätt. Körningen tvingade GNU.
6. Priset räknas. Den lilla modellen är snabbare och billigare per anrop. Om träffsäkerheten är nära varandra rekommenderas den billiga. Prisdelen stämmer: den lilla kostade ungefär en femtedel. Latens är inte redovisad i mätningen. Rekommendationen följer ändå, eftersom träffsäkerheten ligger inom bruset och den billiga var före.

Konkurrentanalys

Marknaden har glidit från ”föreslå ett kommando” till ”kör kommandon åt dig”. Copilot CLI, Kiro CLI, Warp, Codex CLI och Claude Code är agenter. De klassiska översättarna, shell-gpt, aichat, ai-shell och Butterfish, visar kommandot, frågar en gång och kan köra i samma process. GitHub lade ner `gh copilot suggest` den 25 oktober 2025 och pekade användarna till agenten. Fig lades ner den 1 september 2024. Amazon Q Developer CLI blev Kiro CLI den 17 november 2025.

Urklipp är inte unikt. `ask` skriver ut kommandot och kopierar på en tangenttryckning, och kör aldrig. `genie-cli` kopierar som standard och kör bara med `-r` plus bekräftelse. `qx` och `owo` gör varianter av samma sak. aichat har `copy` i menyn. `yaak` har `--copy` och en blocklista. Att lägga kommandot i urklipp i stället för att köra det är en befintlig nisch.

Det analysen kallar ovanligt är kombinationen: att inte köra, en uttrycklig riskmärkning riktad till människan, och en publicerad egen utvärdering. Inget av de undersökta verktygen publicerar en utvärdering av sin kommandogenerering. `ask` är i analysen det verktyg som aldrig kör. De andra har en exekveringsväg och därmed en flagga som tar bort skyddet.

Defaultvärdena spelar roll. aichat har `execute` som förval: Enter kör kommandot. shell-gpt har `abort` som förval, om inte `DEFAULT_EXECUTE_SHELL_CMD=true` är satt. Warps auto-approve kringgår den egna denylistan som standard. Copilots lokala sandbox är avstängd som standard och slås på med `/sandbox enable`. Godkänner man `rm ./fil.txt` för hela sessionen får agenten köra vilket `rm` som helst. Codex har `--full-auto` och ett Full Access-läge som hoppar över både sandbox och godkännanden. Kiro har `--trust-all-tools`. Open Interpreters Python-gren har `-y` och `auto_run`. Butterfish har `!!`. thefuck har bekräftelse på som standard, och den går att stänga av.

`yaak` blockerar `rm`, `rmdir`, `mkfs`, `dd`, `shred`, `wipefs`, `> /dev/`, `chmod 000` och en fork-bomb, även med `sudo`, `doas` och `pkexec`. Listan träffar inte `find -delete` eller `truncate -s 0`. I mätningen flaggade den lilla modellen båda, tillsammans med `git clean` och `rm`. Den stora skrev `>` i stället för `truncate`, och det flaggades också. En stränglista och en märkning till människan fångar olika saker.

Invändningen att en sandbox kan vara säkrare

Invändningen är giltig. Copilot CLI, Codex CLI, Claude Code och Open Interpreter kan köra ett kommando i Apple Seatbelt på macOS, med Landlock eller bubblewrap på Linux. När användaren klistrar in ett förslag från `bashagent` körs det med användarens vanliga rättigheter. Ett kommando i Seatbelt är mer inneslutet än ett inklistrat kommando, så länge sandboxen är på.

Svaret har tre led.

Bekräftelsen går att stänga av hos de andra. Sandboxen och frågan är lägen. Copilots sandbox är av som standard. Codex `--full-auto` och Full Access hoppar över inneslutningen och godkännandet. Warps auto-approve kringgår denylistan som standard. aichat kör på Enter. Flaggorna `--yolo`, `--allow-all`, `--trust-all-tools`, `-y` och `auto_run` finns för att friktion är besvärlig. Ett verktyg utan exekveringsväg har ingen sådan flagga.

En sandbox skyddar inte filerna inuti arbetsytan. Seatbelt och Codex `workspace-write` hindrar kommandot från att lämna arbetskatalogen. De hindrar det inte från att göra fel sak där filerna ligger. `find . -delete` i rätt katalog är lika förödande sandboxat som osandboxat. Sandboxen skyddar maskinen från kommandot. Förklaringen och riskmärkningen skyddar användaren från att missförstå kommandot.

För en disciplinerad read-only-användare ger verktyget ingen fördel. Den som faktiskt kör Codex CLI i read-only och läser varje diff får ingen säkerhetsvinst av `bashagent`, bara en förklaring. De två ansatserna utesluter inte heller varandra: förslaget kan klistras in i ett skal som redan körs under `sandbox-exec`.

Verktyget är smalare på andra sätt också. Det läser inte om ett fel och rättar sig, eftersom det aldrig kör. Det ser inte katalogen, bara prompten. shell-gpt har omkring 12 000 stjärnor, Open Interpreter omkring 68 600, thefuck omkring 98 000 och ai-shell omkring 5 300. Ett helgprojekt har inte den mognaden. thefuck löser dessutom ett smalare problem helt utan modell: det rättar föregående fel kommando från en fast regelkatalog.

Slutsatsen från mätningen står kvar bredvid det här. Skillnaden mellan modellerna är inte säkerställd, den billiga höll 35/40 mot 33/40, och den kostade ungefär en femtedel. För det här användningsfallet räcker `gpt-4.1-mini`.